

Assignment 2

Table of Contents

Calculation of trusses.....	1
How it was calculated:.....	3
printed results from calculations:.....	8

Calculation of trusses

Determine the force in each member of the truss shown schematically in Figure 1. Clearly identify if the members are in Tension [T] or in Compression [C].

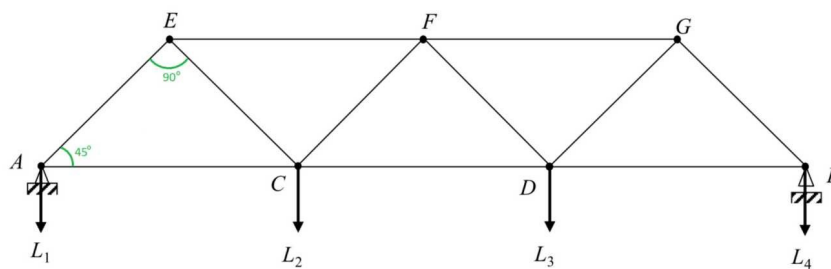


Figure 1: Schematic representation of a bride (planar truss). Note: assume that both A and B are roller supports.

Each student must use the loading situation stated in Table 1. Notice that the values of L1, L2, L3, and L4 (in g) vary for each of you, so please clearly identify the ones that you have used in the calculations. For example, student 1 will calculate the truss with the following loads: L1 = 250 g, L2 = 100 g, L3 = 150 g, L4 = 300 g. Student 2: L1 = 100 g, L2 = 250 g, L3 = 1000 g, L4 = 500 g, and Student 3: L1 = 500 g, L2 = 500 g, L3 = 100 g, L4 = 250 g.

Table 1: Values of the loads [g] for the calculations

Student	Mass [g]			
	L1	L2	L3	L4
1	125	50	75	150
2	50	125	500	250
3	250	250	50	125
4	550	362.5	150	325
5	650	325	50	300
6	100	475	325	325
7	75	375	425	600

8	150	75	475	225
9	325	225	575	225
10	500	300	475	650
11	325	150	425	675
12	725	625	175	475
13	575	500	250	125
14	575	425	250	725
15	275	200	125	375
16	625	575	175	25
17	125	450	675	175
18	50	125	275	475
19	475	300	400	450
20	750	375	125	575

Stu.	Mass [g]			
	L1	L2	L3	L4
21	475	475	750	700
22	200	75	450	550
23	575	200	50	250
24	375	325	350	600
25	50	125	375	350
26	100	100	475	600
27	400	375	450	575
28	700	425	700	425
29	750	175	575	425
30	425	50	200	50
31	750	50	125	350
32	75	475	250	650
33	300	325	725	450
34	450	400	550	450
35	25	325	525	725
36	725	300	375	625
37	700	425	375	250
38	600	375	525	300

	39	475	125	250	175
	40	475	125	250	200
	Stu.	Mass [g]			
		L1	L2	L3	L4
	41	100	75	325	100
	42	150	75	125	100
	43	200	500	150	175
	44	250	100	100	175
	45	125	125	500	200
	46	175	150	550	300
	47	225	275	250	75
	48	75	75	350	100
	49	275	275	650	450
	50	450	650	550	100
	51	50	325	525	725

Please, enter the results in the following table (you would need it in lab1), and submit it together with your calculations in a single pdf.

Student number 7

How it was calculated:

```
from sympy.core.relational import Equality, Relational, Unequality
import sympy as sp
from sympy import Eq, Matrix, Symbol, cos, rad, sin, symbols
```

```
sp.init_printing(use_unicode=True)
```

```
class Member2D:
    """Represents a 2D axial force member or external force with a scalar magnitude and a
    direction angle in degrees."""
    def __init__(self, name: str, angle_deg: float = 0, is_scalar: bool = True):
        self.name = name
        self.angle_deg = angle_deg
        theta = rad(angle_deg)

        if is_scalar:
            # Single scalar magnitude (standard truss member force)
            self.magnitude = symbols(name)
            self.x = self.magnitude * cos(theta)
            self.y = self.magnitude * sin(theta)
        else:
            # Known vector component input
            self.x = symbols(f"{name}_x")
```

```
self.y = symbols(f"{{name}}_y")
```

```
def toMatrix(self):  
    return Matrix([self.x, self.y])
```

```
def __add__(self, other):  
    if isinstance(other, Member2D):  
        return VectorSum(self.toMatrix() + other.toMatrix())  
    return VectorSum(self.toMatrix() + other)
```

```
def __radd__(self, other):  
    return self.__add__(other)
```

```
def __sub__(self, other):  
    if isinstance(other, Member2D):  
        return VectorSum(self.toMatrix() - other.toMatrix())  
    return VectorSum(self.toMatrix() - other)
```

```
def __neg__(self):  
    return VectorSum(-self.toMatrix())
```

```
class VectorSum:  
    """Helper wrapper to allow fluent addition/subtraction of 2D matrices."""
```

```
def __init__(self, mat: Matrix):  
    self.mat = mat
```

```
def __add__(self, other):  
    if isinstance(other, Member2D):  
        return VectorSum(self.mat + other.toMatrix())  
    return VectorSum(self.mat + getattr(other, "mat", other))
```

```
def __sub__(self, other):  
    if isinstance(other, Member2D):  
        return VectorSum(self.mat - other.toMatrix())  
    return VectorSum(self.mat - getattr(other, "mat", other))
```

```
def toMatrix(self):  
    return self.mat
```

```
# Define Member Forces (1 scalar unknown per member)
```

```
N_AC = Member2D("N_{{AC}}", angle_deg=0)
```

```
N_CD = Member2D("N_{{CD}}", angle_deg=0)
```

```
N_BD = Member2D("N_{{BD}}", angle_deg=0)
```

```
N_EF = Member2D("N_{{EF}}", angle_deg=0)
```

```
N_FG = Member2D("N_{{FG}}", angle_deg=0)
```

```
N_A = Member2D("N_{{A}}", angle_deg=90)
```

```
N_B = Member2D("N_{{B}}", angle_deg=90)
```

```
N_AE = Member2D("N_{{AE}}", angle_deg=45)
```

```
N_CE = Member2D("N_{{CE}}", angle_deg=135)
```

```
N_CF = Member2D("N_{{CF}}", angle_deg=45)
```

```
N_DF = Member2D("N_{{DF}}", angle_deg=135)
```

```
N_DG = Member2D("N_{DG}", angle_deg=45)
N_BG = Member2D("N_{BG}", angle_deg=135)
```

```
N_L1 = Member2D("N_{L1}", angle_deg=-90)
N_L2 = Member2D("N_{L2}", angle_deg=-90)
N_L3 = Member2D("N_{L3}", angle_deg=-90)
N_L4 = Member2D("N_{L4}", angle_deg=-90)
```

```
unknown_symbols = [
symbols("N_{AC}"),
symbols("N_{CD}"),
symbols("N_{BD}"),
symbols("N_{EF}"),
symbols("N_{FG}"),
symbols("N_{AE}"),
symbols("N_{CE}"),
symbols("N_{CF}"),
symbols("N_{DF}"),
symbols("N_{DG}"),
symbols("N_{BG}"),
symbols("N_{A}"),
symbols("N_{B}"),
]
```

```
E: Equality | Relational | Unequality = Eq((N_EF-N_CE-N_AE).toMatrix(), Matrix([0,0]))
F: Equality | Relational | Unequality = Eq((N_FG-N_EF-N_DF-N_CF).toMatrix(), Matrix([0,0]))
G: Equality | Relational | Unequality = Eq((-N_FG-N_DG-N_BG).toMatrix(), Matrix([0,0]))
```

```
# Known Loads (Downward vertical forces)
g:float = 9.81
```

```
L1:float=75e-3
L2:float=375e-3
L3:float=425e-3
L4:float=600e-3
Navn:str="Lars Birger"
```

```
# Substitute known load values
sub_loads = {
symbols("N_{L1}"): L1 * g,
symbols("N_{L2}"): L2 * g,
symbols("N_{L3}"): L3 * g,
symbols("N_{L4}"): L4 * g,
}
```

```
# Joint Equilibrium Equations (Sum of forces = 0 at each node)
A: Equality | Relational | Unequality = Eq((-N_A+N_L1 + N_AC +
N_AE).toMatrix().subs(sub_loads), Matrix([0, 0]))
B: Equality | Relational | Unequality = Eq((-N_B+N_L4 + N_BG +
N_BD).toMatrix().subs(sub_loads), Matrix([0, 0]))
C: Equality | Relational | Unequality = Eq((N_L2 + N_CD + N_CE + N_CF -
N_AC).toMatrix().subs(sub_loads), Matrix([0, 0]))
D: Equality | Relational | Unequality = Eq((N_L3 + N_DF + N_DG - N_CD -
```

```
N_BD).toMatrix().subs(sub_loads), Matrix([0, 0]))
```

```
# move from vector to scalar functions
```

```
vector_eqs: list[Equality | Relational | Unequality] = [A, B, C, D, E, F, G]
```

```
scalar_eqs: list[Equality | Relational | Unequality] = []
```

```
unit_vector = [  
symbols("A_x"),  
symbols("A_y"),  
symbols("B_x"),  
symbols("B_y"),  
symbols("C_x"),  
symbols("C_y"),  
symbols("D_x"),  
symbols("D_y"),  
symbols("E_x"),  
symbols("E_y"),  
symbols("F_x"),  
symbols("F_y"),  
symbols("G_x"),  
symbols("G_y"),  
]
```

```
for eq in vector_eqs:
```

```
scalar_eqs.append(Eq(eq.lhs[0], eq.rhs[0])) # Sum of Fx = 0
```

```
scalar_eqs.append(Eq(eq.lhs[1], eq.rhs[1])) # Sum of Fy = 0
```

```
solution = sp.solve(scalar_eqs, unknown_symbols, dict=True)[0]
```

```
for variable, value in solution.items():
```

```
print(f"The force of {variable}\tis\t{value:+3.5f}[N]")
```

```
print("\nNegative means rod 'N_{{a}}[b]] is in tension and is pulling in")
```

```
print("positive means rod 'N_{{a}}[b]] is in compression and is pushing out")
```

```
print(f"\nDette er for vektene til {Navn}, som er:")
```

```
print(f"\tL1: {{L1*1e3:#.2f}}\t[g]\n\tL2: {{L2*1e3:#.2f}}\t[g]\n\tL3: {{L3*1e3:#.2f}}\t[g]\n\tL4:  
{{L4*1e3:#.2f}}\t[g]")
```

```
import numpy as np
```

```
c=np.cos(np.deg2rad(45))
```

```
A_x = solution[symbols("N_{{AC}}")] + solution[symbols("N_{{AE}}")] * np.cos(np.deg2rad(45))
```

```
A_y = L1 * g + solution[symbols("N_{{AE}}")]*c
```

```
B_x = solution[symbols("N_{{BD}}")] + solution[symbols("N_{{BD}}")]*c
```

```
B_y = solution[symbols("N_{{B}}")] + solution[symbols("N_{{BD}}")]*c
```

```
Btot = ((B_x**2) + (B_y**2))**(1/2)
```

```
print(f"A_x:\t{A_x:+0.2f}")
```

```
print(f"A_y:\t{A_y:+0.2f}")
```

```
print(f"B_x:\t{B_x:+0.2f}")  
print(f"B_y:\t{B_y:+0.2f}")  
print(f"B: \t{Btot:+0.2f}")
```

printed results from calculations:

The force of $N_{\{AC\}}$ is +3.84225[N]
The force of $N_{\{AE\}}$ is -5.43376[N]
The force of $N_{\{A\}}$ is -4.57800[N]
The force of $N_{\{BD\}}$ is -4.00575[N]
The force of $N_{\{BG\}}$ is -5.66499[N]
The force of $N_{\{B\}}$ is -9.89175[N]
The force of $N_{\{CD\}}$ is +7.84800[N]
The force of $N_{\{CE\}}$ is +5.43376[N]
The force of $N_{\{CF\}}$ is -0.23122[N]
The force of $N_{\{DF\}}$ is +0.23122[N]
The force of $N_{\{DG\}}$ is +5.66499[N]
The force of $N_{\{EF\}}$ is -7.68450[N]
The force of $N_{\{FG\}}$ is -8.01150[N]

Negative means rod ' $N_{[a][b]}$ ' is in tension and is pulling in
positive means rod ' $N_{[a][b]}$ ' is in compression and is pushing out

Dette er for vektene til Lars Birger, som er:

L1: 75.00 [g]

L2: 375.00 [g]

L3: 425.00 [g]

L4: 600.00 [g]

A_x : +0.00

A_y : -3.11

B_x : -6.84

B_y : -12.72

B : +14.45